Rapport Général d'Itération – Groupe CD3

Projet : Gravity BOT.

Introduction et Objectifs de l'itération

L'objectif principal de cette itération a été de poser des fondations robustes pour les deux piliers du jeu : un moteur physique mathématiquement fiable (déplacements, collisions, forces) et une interface graphique fluide (menus, éditeur, rendu de la grille). L'équipe a travaillé en parallèle sur la logique métier, l'expérience utilisateur et l'assurance qualité.

1. Interface Graphique et Expérience Utilisateur (Java Swing)

Le développement de l'interface graphique a permis de structurer la navigation et le rendu visuel du jeu :

- **Navigation et Menus :** Implémentation du `MenuPrincipal` avec un système d'onglets internes (`CardLayout`) pour basculer de manière fluide entre l'Accueil, les Paramètres (volume, contraste avec pop-ups `JOptionPane`) et les Crédits aux couleurs du jeu (Bleu Néon sur fond Espace).
- **Fenêtres de Jeu et Éditeur :** Développement de `FenetreJeu` (bandeau d'informations, boutique, contrôles) et `ModeEditeur` (outils de construction, slider de budget).
- **Moteur de Rendu Visuel :** Création de `AfficherGrille` utilisant un thème "Radar" (fond sombre, grille bleue transparente). Les entités sont visuellement différenciées (le satellite est un cercle blanc, les obstacles des carrés orange/rouge).
- **Audio :** Mise en place d'un `GestionnaireAudio` pour contrôler le volume en temps réel.

2. Noyau Moteur et Utilitaires Mathématiques

L'architecture centrale du jeu a été conçue pour garantir la stabilité de la simulation physique :

- **Bibliothèque Mathématique :** Refonte et sécurisation de la classe `Vecteur`. Ajout d'opérations spatiales avancées : distance euclidienne, projection orthogonale, calcul de vecteurs normaux et rotation autour d'un pivot. Sécurisation par le passage d'objets (encapsulation) pour éviter les effets de bord.
- **Architecture des Entités :** Implémentation de la classe abstraite `Acteur`, qui encapsule les positions vectorielles et impose le contrat `actualiser(double dt)`. Cette classe a été dérivée en `Obstacle` et `Projectile`.
- **Boucle de Jeu (Game Loop) :** Initialisation des classes `Map` et `Game` pour assurer l'actualisation fluide des éléments à chaque *frame* et gérer l'environnement global.

3. Système de Collisions et Mécaniques de Jeu

La détection des impacts et l'application des forces ont été entièrement développées :

- **Primitives de Collision :** Spécification de l'interface de `Collision` et implémentation de multiples primitives géométriques : `CollisionCirculaire`, `CollisionRectangle`, et `CollisionLineaire`.
- **Obstacles Actifs :** Création d'obstacles dynamiques exploitant le polymorphisme.
    - *Obstacles Circulaires :* Simulation physique des rebonds.
    - *Obstacles Propulseurs (Ventilateur) :* Application d'une force directionnelle continue sur le projectile, avec un typage défensif (clonage des vecteurs de force).
    - *Obstacles Magnétiques :* Systèmes d'attraction.

4. Assurance Qualité et Tests (JUnit)

Une part importante de l'itération a été allouée à la fiabilisation du code pour garantir la stabilité de la branche principale :

- **Tests du Projectile :** Validation des calculs de translation dans le temps (facteur `dt`), vérification de l'indépendance des variables (encapsulation), et validation des lois physiques (bridage et normalisation de la vitesse maximale).
- **Tests des Collisions :** Exécution d'une suite exhaustive de tests couvrant les cas nominaux, limites et négatifs.
    - 10 tests pour la détection rectangulaire.
    - 6 tests pour la détection circulaire et les rebonds.
    - Tests géométriques stricts sur les collisions linéaires (projectile dans la zone, hors portée, hors épaisseur, direction opposée).
- **Débogage :** Traque et correction systématique des erreurs de compilation et de logique (notamment sur `CollisionRectangle`). Tous les tests sont actuellement passants (100% de succès).